

Handoff — what's done, what's left, and exactly how to finish it

Written for someone picking this up cold. Read this, then work top to bottom.

Branch: `iterations` (8 commits ahead of `main`). Everything below is on that branch. Merging it into `main` is the last task, not the first.

```
git checkout iterations && npm install && npm test && npm run dev
```

44 tests should pass. The app runs at `localhost:3000` with no configuration.

1. What this project is, in four sentences

Rain lets you bound **how much** an agent spends and **where**. Mandate checks **why** — the agent must declare the purchase order it negotiated, deterministic code checks that against the real record, and if it doesn't hold, **no card is ever issued**. Not a decline: there is nothing to decline, because the instrument was never created.

The differentiator is **replay**: because rules are versioned data and no model sits in the decision path, you can edit a rule and re-run all history against it.

2. Current state

Working and tested:

7 deterministic checks	pure functions, no I/O, no model, no wall clock
Rules as versioned data	sha256 per version, each citing its real-world control
Append-only decision log	stores a record <i>snapshot</i> , not a pointer
Replay + rule diff	"across 47 decisions, 8 approvals would now be refused"
Negotiation	4 sellers, distinct strategies, one counter-offer round
Human oversight	above \$25k a purchase is held ; a named person releases it
Dual control	a policy change needs a second person; the author cannot self-approve
Idempotency under concurrency	order line reserved before issuance; race verified closed
Card lifecycle	issue → settle → revoke
Monad anchor	written and wired, inert until credentials exist

Simulated, not real: cards. Everything else around them is real.

3. 🚫 Do these first — they are blocking, and none are code

3a. Get a database (3 minutes, highest priority)

Without `DATABASE_URL` the deployed app loses its decision log on every cold start and **replay silently breaks on the exact URL you submit**. It works perfectly on a laptop either way, which is what makes it dangerous.

Free Postgres at neon.tech → copy the connection string → Vercel → Settings → Environment Variables → `DATABASE_URL`. See [DEPLOY.md](#).

The app shows a red banner in production if this is missing. Do not ship with it showing.

3b. Ask a Rain engineer three questions

All three are one-line answers that unblock finished code. Detail in [RAIN-API-CONFIRMED.md](#).

1. **The collateral contract on our sheet isn't linked to our user.** `GET /issuing/users/{userId}/contracts` returns `[]` and `GET /contracts/{id}` returns 403. Does it need attaching, or funding with RUSD? *Without this, issued cards have no spending power — this is the one thing between us and money genuinely moving.*
2. **What status value deactivates a card?** `PATCH /issuing/cards/{id}` exists, but `{"status":"inactive"}` returns 400. Put the answer in `RAIN_CARD_INACTIVE_STATUS` and revocation works with no code change.
3. **What goes in configuration to set a spending limit and short expiry?** It comes back as `{"currency":"usd"}`. Rain's *minimum* card body produces an active card with **no limit and a six-year expiry** — we must always send an explicit limit.

3c. Monad credentials

A testnet RPC URL and a key with testnet MON for gas → `MONAD_RPC_URL` and `MONAD_PRIVATE_KEY`. The anchor button appears automatically and the code is done.

3d. Delete one comment

[lib/money.ts](#) line 1 references `crossval-pricing/lib/money.ts`. On a public repo judged under "all code written fresh today", that reads as copied from another project. **Delete or reword that line before submitting.**

3e. One stray card on the Rain account

`ab3ea8c1-b0f3-4409-a7fb-a351e6a4d3ce` — active, unscoped, expires 2032, created accidentally while mapping the API. No collateral behind it, no real money. Deactivate it once 3b#2 is answered, **or** keep it deliberately as the demo's foil: the unscoped card next to a Mandate card scoped to one PO and expiring with its quote.

4. Remaining build work, in priority order

Each is self-contained. Do them in order; stop whenever it's good enough.

Task 1 — Merge `iterations` into `main` ★ do this once the above is stable

```
git checkout main && git pull
git merge iterations
npm test && npm run build
git push
```

Expect conflicts in `app/page.tsx`, `components/*`, `tailwind.config.ts` — the two of us have been editing the same UI. **Resolution rule: keep A's light-theme styling and Mandate's newer functionality.** If a component on `main` looks visually newer but is missing props like `onAnchor`, `ruleChanges`, `onPropose`, take `iterations`' logic and re-apply `main`'s classNames.

Task 2 — Multi-agent live view

The last unbuilt item from `ALTERNATIVE-IDEAS.md`. A "Run all agents" button that fires every task in sequence with the pipeline diagram animating and budgets visibly draining.

- Add to `components/RunPanel.tsx`, next to "Reset demo".
- Call the existing `run()` in `app/page.tsx` for each task in turn, awaiting each so the stage trace stays readable.
- **Do not** run them concurrently on the same PO — that path is deliberately refused now (see iteration 7).

Task 3 — Show the control basis in the rule editor

Every rule already carries a `basis` string naming the real control it implements (three-way match, idempotency key, delegation of authority). It is in the data and in the docs but **not yet on screen**.

- `components/RuleEditor.tsx`, under each rule label, small and muted.
- This is cheap and it is most of the trust argument. High value per minute.

Task 4 — Per-decision Monad anchoring (optional)

`lib/monad/anchor.ts` already does the work; point it at a decision hash instead of a rule hash. **Cut this before anything else** — the rule-version anchor is the structural one.

Task 5 — Rehearse

Genuinely the highest-value remaining activity. The demo order is in `SUBMISSION.md §4`. Time it. The run-it-twice moment (step 2) and the self-approval refusal (step 6) are the two that land.

5. Where everything lives

<code>lib/checks/</code>	the 7 checks + <code>verify()</code> . Pure. No I/O, no model, no clock.
<code>lib/rules/</code>	versioned rule data, sha256, diff between versions
<code>lib/replay/</code>	re-judging stored decisions against another rule version
<code>lib/store/</code>	append-only log. Postgres when <code>DATABASE_URL</code> is set, memory otherwise
<code>lib/pipeline.ts</code>	the stages in order — this file IS the architecture diagram
<code>lib/monad/anchor.ts</code>	publishing a rule version's hash to Monad testnet
<code>lib/rain-client.ts</code>	everything that talks to Rain, in one place
<code>lib/rain/issuer.ts</code>	the seam between a passing <code>verify()</code> and a real card
<code>lib/negotiation.ts</code>	competing sellers (A's)
<code>app/api/</code>	state, run, purchase, approve, rules, rules/activate, replay, anchor, reset
<code>docs/CHANGELOG.md</code>	what changed and why, newest first — keep adding to it

6. Invariants — do not break these

These are the reasons the project is defensible. Each one is load-bearing.

1. **No model in `lib/checks/`.** Not a safety preference — replay is only meaningful because the same inputs always give the same answer. An LLM here destroys the headline feature.
2. **No wall clock in the checks.** Time comes from `record.observedAt`. Reading `Date.now()` would make a decision replayed tomorrow get judged against tomorrow.
3. **Rain is only ever reached after `verify()` returns ok.** Enforced by the shape of `runPipeline`, not by a flag. A refusal must produce *no instrument*.
4. **Decisions store a snapshot, never a pointer.** Replaying a pointer re-judges today's facts, which is a new decision, not a replay.
5. **Nothing is updated in place.** A hold and its release are two rows. A rule edit is a new version.
6. **Never claim something happened that didn't.** Simulated cards say *simulated*; a card that couldn't be retired reports a failed stage. This is worth more than a smoother demo.

7. The pitch, if you need it cold

Rain gives an agent a card with a limit and an allowlist — that bounds **how much** and **where**. Nothing bounds **why**. An agent can buy the wrong thing at the right price from an allowed supplier and every control passes. So we make the agent declare the purchase order it negotiated, and we check it against the real record. If it doesn't hold, **no card is issued** — not a decline, because there's nothing to decline. Above a limit you set, a named person releases it. Changing the rules takes two people. And because there's no AI in the decision path, we can re-run every past decision against a new rule and trust the difference.